Swin-Tiny_Swin_V2_LBP_GLCM_Gabor_Wavelet_Kod_Aciklamasi.pdf - local repository text copy

EXPERIMENT 02

CODE DESIGN AND IMPLEMENTATION REPORT

Eye Region Deepfake Detection with Swin V2-Tiny and LBP, GLCM, Gabor, Wavelet Feature Fusion

Technical explanation of classes, functions, hyperparameters, training loop, checkpointing and quality controls

Prepared from: Experiment_02_Eye_SwinV2_TextureFusion (1).ipynb

Date: 6 August 2026

EXPERIMENT 02 | EYE SWINV2 + TEXTURE FUSION | CODE DESIGN REPORT

1. Purpose and Scope of the Report

This report explains the uploaded Jupyter/Colab notebook as a software and machine-learning system rather than merely listing its cells. It clarifies why each major class, function and hyperparameter exists, how data flows through the pipeline, why training is performed across epochs, how the model combines learned and handcrafted representations, and how reproducibility and experiment safety are protected.

The notebook implements binary real/fake classification using combined eye-region images. The deep-learning branch uses an ImageNet-pretrained Swin V2-Tiny backbone, while the handcrafted branch extracts LBP, GLCM, Gabor and wavelet descriptors. Both branches are projected to a common hidden space and fused before binary classification.

Figure 1. End-to-end execution flow of the notebook.

2. Configuration-based Experiment Design

The CONFIG dictionary centralizes all paths, feature settings, model settings and training parameters. This design prevents values from being scattered across cells and makes the experiment easier to reproduce, audit and modify. A resolved YAML copy is saved into the run directory so that every result remains linked to the exact configuration that produced it.

Parameter	Reason / Function
image_size = 224	Matches the standard input resolution expected by many pretrained vision backbones and keeps memory usage manageable.
model_name = swin_v2_t	Documents the intended backbone; however, the constructor directly calls swin_v2_t, so this key is descriptive rather than dynamically used.
pretrained = True	Starts from ImageNet-learned visual representations instead of training a transformer from random weights on a relatively limited dataset.
freeze_backbone_epochs = 5	Protects pretrained representations during the first epochs while the new fusion and classifier layers adapt.
batch_size = 16	Balances GPU memory demand, gradient stability and iteration speed for 224x224 images plus texture vectors.

EXPERIMENT 02 | EYE SWINV2 + TEXTURE FUSION | CODE DESIGN REPORT

Parameter	Reason / Function
epochs = 30	Defines the maximum number of complete passes over the training set; early stopping may terminate earlier.
learning_rate = 2e-4	Used for newly initialized projection/classifier layers that require faster adaptation.
backbone_learning_rate = 2e-5	Uses a ten-times smaller rate for pretrained backbone parameters to reduce destructive updates.
weight_decay = 1e-4	Regularizes weights through AdamW and helps

reduce overfitting.

dropout = 0.30 Randomly disables 30% of activations during
training to improve generalization.
hidden_dim = 256 Maps both branches into an equally sized latent
representation before fusion.
patience = 8 Stops training after eight consecutive epochs
without validation ROC-AUC improvement.
threshold search 0.10-0.90, 161 steps Searches validation thresholds in increments of
0.005 to maximize F1.
amp = True Enables automatic mixed precision on CUDA for
faster training and lower memory use.
gradient_clip_norm = 1.0 Limits gradient norm to reduce instability and
exploding gradients.
seed = 42 Fixes random generators to support repeatable
data order and parameter initialization behavior.
3. Why Epochs Are Used
An epoch is one complete traversal of the training dataset. A neural network does not learn the real/fake

distinction in a single pass. During each mini-batch, the model calculates an error, backpropagates that
error and slightly updates its parameters. Repeating this process over multiple epochs allows the model
to
progressively refine its representation of eye-region artifacts.
The code sets a maximum of 30 epochs, not a mandatory 30-epoch run. Training can stop earlier through
patience-based early stopping. This combination gives the model enough opportunity to learn while
protecting it from unnecessary computation and overfitting.
Epoch-stage mechanism Why it is used
Epochs 1-5: frozen backbone
Only projection and classifier layers learn initially.
This stabilizes transfer learning because randomly
initialized layers adapt before pretrained visual
filters are modified.
Epoch 6 onward: final Swin stage unfrozen
The last feature stage, final normalization and
Swin projection are fine-tuned for eye-region
deepfake cues while earlier generic visual features
stay frozen.
Validation after every epoch Measures generalization on unseen validation data

Epoch-stage mechanism Why it is used
rather than trusting training performance.
Best checkpoint by validation ROC-AUC Chooses the model that best ranks real versus
fake samples independently of a fixed threshold.
Per-epoch threshold search Finds the decision boundary that maximizes
validation F1 for the current model.
Early stopping after patience=8 Stops when validation ranking no longer improves,
reducing overfitting and wasted GPU time.
CosineAnnealingLR
Gradually reduces the learning rate over epochs,
allowing coarse updates early and finer
adjustments later.
4. Reproducibility, Run Identity and Atomic Output Writing
4.1 seed_everything()
```
def seed_everything(seed: int) -> None:
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True
```
This function synchronizes the main random number generators. Deterministic cuDNN behavior is
preferred over maximum speed because experiment comparability is more important than a small
performance gain. A timestamped RUN_ID prevents outputs from different runs from overwriting one
another.
4.2 Atomic write helper functions
The atomic_write_* functions first write to a temporary file, flush the bytes to disk, validate the
temporary
artifact where appropriate, and then replace the final target with os.replace. This pattern protects CSV

files,

feature caches and checkpoints from becoming half-written if Colab disconnects or local source package temporarily fails.

Function	Role
atomic_write_bytes / text	Base layer for crash-resistant text and byte output.
atomic_write_json	Writes metrics and audit data in readable structured form.
atomic_write_csv	Serializes DataFrames in memory before committing them.
atomic_joblib_dump	Saves and reloads the feature cache before replacing the final cache file.
atomic_torch_save	Saves a checkpoint, reloads it and verifies required keys before committing it.

5. Metadata Loading and Quality Gates

The metadata section normalizes label, split and status strings; filters rows with valid eye-image paths and accepted labels/splits; maps real to 0 and fake to 1; resolves file paths; removes missing files and duplicate sample IDs; and checks that source videos do not overlap across train, validation and test sets.

The video-level disjointness assertions are particularly important. Multiple frames from the same video are visually correlated. If frames from one source video appear in both training and testing, the model may memorize video-specific patterns and produce an artificially high score. The assertions therefore function as a leakage barrier.

6. Handcrafted Texture Feature Extraction

The handcrafted branch is intended to complement Swin V2. Deep features can learn global and semantic patterns, while handcrafted descriptors explicitly summarize local texture, co-occurrence, orientation-frequency responses and multiscale frequency content. These cues are relevant because synthesis and compression artifacts may alter fine eye-region texture even when the image appears visually realistic.

Function	Detailed purpose
read_gray_image	Reads each ROI in grayscale and resizes it to the configured shape. Grayscale is sufficient because these texture algorithms primarily analyze intensity structure.
safe_entropy	Computes histogram entropy while safely handling empty or zero-mass inputs.
extract_lbp	Builds normalized uniform-LBP histograms at radii 1, 2 and 3. LBP describes local micro-patterns such as flat areas, edges and spots.
extract_glcm	Quantizes intensity to 32 levels and computes six co-occurrence properties over three distances and four angles. It represents spatial relationships between neighboring intensities.
extract_gabor	Applies filters at three frequencies and six orientations, then records mean, standard deviation, energy and entropy. This captures oriented and frequency-selective texture.
coefficient_statistics	Provides a reusable six-statistic summary for wavelet coefficient matrices.
extract_wavelet	Uses db2 wavelets at level 2 to summarize approximation and horizontal/vertical/diagonal detail coefficients at multiple scales.
extract_texture_features	Runs all four extractors, records group slices and feature names, concatenates the vector and rejects NaN/Inf values.

Based on the configuration, the resulting full texture vector contains 240 values: 54 LBP bins, 72 GLCM values, 72 Gabor statistics and 42 wavelet statistics. Group slices are preserved so ablation modes can select only specific feature families without recomputing the cache.

7. Feature Cache and Train-Only Scaling

7.1 Why a persistent feature cache is used

LBP, GLCM, Gabor and wavelet extraction is CPU-intensive. Recomputing the same 240-dimensional vector in every epoch would make training unnecessarily slow. The cache computes each sample once and stores the vector by sample_id. A SHA-256-derived cache version changes when feature settings change, preventing incompatible caches from being reused.

The cache is flushed every 100 processed samples. This interval limits work lost during interruptions without writing to local source after every image.

7.2 Why StandardScaler is fitted only on training data

Texture features have very different numerical ranges. StandardScaler centers and scales each dimension so that large-magnitude variables do not dominate the texture projection. It is fitted only on the training split

because fitting on validation or test values would leak information about held-out distributions into training.

8. EyeFusionDataset and DataLoader Design

8.1 Why EyeFusionDataset was written

EyeFusionDataset is the bridge between tabular metadata, image files, cached texture vectors and PyTorch. A custom Dataset is necessary because every sample has two synchronized inputs - an RGB image tensor and a selected texture vector - plus target and traceability metadata.

Method / field	Function
__init__	Stores a clean DataFrame copy, current ablation mode, image transform and training-fitted scaler.
__len__	Returns the number of available samples so DataLoader can iterate correctly.
__getitem__	Loads the image, converts it to RGB, normalizes it, retrieves and scales texture features, selects the required feature groups and returns tensors plus identifiers.
sample_id / video_id / image_path	Maintains auditability so every prediction can be traced back to its original sample and source video.

8.2 Why geometric augmentation is not applied

The notebook deliberately uses identical resize-normalize transforms for training and evaluation. Texture

vectors are extracted from the original image and cached. If the RGB image were randomly flipped, rotated

or cropped while the cached texture vector still described the unmodified image, the two branches would represent different versions of the sample. Avoiding geometric augmentation preserves branch alignment. A future implementation could apply augmentation before feature extraction or recompute texture features online.

8.3 DataLoader choices

Training uses shuffle=True so batches do not repeatedly follow metadata order. Validation and test use shuffle=False for deterministic evaluation and easier tracing. A seeded torch.Generator improves repeatability. pin_memory=True can accelerate CPU-to-GPU transfer, and num_workers=2 performs moderate parallel loading without overwhelming Colab resources.

9. SwinTextureFusion Model Class

SwinTextureFusion is written as a single configurable class so the same implementation can train deep-only, texture-only and fusion ablations. This avoids maintaining separate models whose differences could introduce unfair comparisons.

Component	What it does and why
Swin V2-Tiny backbone	Extracts hierarchical deep visual features using shifted-window attention. The original classification head is replaced by Identity so its feature vector can be fused.
swin_projection	LayerNorm stabilizes transformer features; Linear maps them to hidden_dim=256; GELU adds nonlinearity; Dropout regularizes.
texture_projection	Applies the same dimensional target and nonlinear processing to the handcrafted vector, placing both branches in comparable latent spaces.
torch.cat(branches, dim=1)	Performs feature-level fusion by concatenating

deep and handcrafted representations.
classifier

Progressively maps the fused vector from 512 or
256 dimensions to a single logit through
BatchNorm, GELU and Dropout.
single output logit
Matches binary BCEWithLogitsLoss; sigmoid is
applied only for probabilities during
metrics/prediction.
freeze_backbone Temporarily prevents pretrained Swin weights from
updating.
unfreeze_backbone_tail
Fine-tunes only the final Swin feature stage, final
norm and projection to control overfitting and
computation.

9.1 Why LayerNorm and BatchNorm are both present
LayerNorm is natural for transformer and vector representations because it normalizes within each sample.

BatchNorm1d is used later in the shared classifier to stabilize the fused hidden activations across the
mini-
batch. They operate at different points and serve complementary roles.
10. Loss, Optimizer, Scheduler and Threshold Selection
10.1 BCEWithLogitsLoss with positive-class weighting
Binary cross-entropy is appropriate because each sample has one binary target. BCEWithLogitsLoss combines sigmoid and cross-entropy in a numerically stable operation. pos_weight is calculated from the training labels so class imbalance does not make the majority class dominate the loss.
10.2 AdamW with two learning rates
build_optimizer separates trainable backbone parameters from newly initialized head parameters. The pretrained backbone receives 2e-5 while the fusion/classifier layers receive 2e-4. AdamW decouples weight decay from gradient updates and is commonly suitable for transformer fine-tuning.
10.3 Why validation F1 threshold search is needed
The network outputs a probability-like score after sigmoid, but converting that score to real/fake
requires a
threshold. A fixed 0.50 threshold is not always optimal, especially under imbalance or asymmetric score calibration. find_best_threshold tests 161 values and selects the one with maximum validation F1. The
test
set is never used to tune this threshold.
10.4 Why ROC-AUC selects the best checkpoint
ROC-AUC evaluates ranking quality across all possible thresholds. Selecting checkpoints by validation ROC-AUC therefore avoids tying model selection to one arbitrary classification threshold. The separately optimized validation threshold is stored in the best checkpoint and then used on the test set.
11. run_epoch(): Training and Evaluation Mechanics
```
with torch.autocast(device_type=DEVICE.type, enabled=CONFIG["amp"] and DEVICE.type == "cuda"):
    logits = model(images, textures)
    loss = criterion(logits, targets)
if training:
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
```
run_epoch is shared by training and validation to avoid duplicated logic. model.train(training) toggles dropout and normalization behavior. During training, gradients are cleared, mixed-precision forward computation is performed, the loss is checked for NaN/Inf, scaled backpropagation is executed, gradients are unscaled and clipped, and optimizer parameters are updated. During validation, the same forward path runs without optimizer steps.
Mechanism Reason
optimizer.zero_grad(set_to_none=True) Clears previous gradients efficiently and reduces
memory writes.

Mechanism Reason
autocast + GradScaler Uses lower precision where safe while scaling loss
to protect small gradients from underflow.
finite loss check Stops immediately when numerical instability
produces NaN or infinity.
gradient clipping Restricts unusually large updates that could

destabilize transformer fine-tuning.
sigmoid only for metrics Keeps the training loss numerically stable by

passing raw logits to BCEWithLogitsLoss.
12. Smoke Test, Checkpoints and Full Training Control
12.1 Why a smoke test is required
The smoke test runs only two training and two validation batches, saves and reloads a checkpoint, and then releases memory. Its purpose is not to estimate accuracy. It detects shape mismatches, invalid feature dimensions, unreadable data, incompatible devices and checkpoint serialization errors before a long training job begins.
12.2 What is stored in a checkpoint
Checkpoint field Why it matters
epoch Identifies the exact training position.
model_state_dict Stores learned model parameters.
optimizer_state_dict Preserves momentum and adaptive optimizer statistics.
scheduler_state_dict Preserves learning-rate schedule progress.
scaler_state_dict Preserves AMP gradient-scaler state.
best_metric_score / best_threshold Preserves model-selection evidence and decision boundary.
history / config / mode / feature_slices Makes the checkpoint self-describing and auditable.
Python, NumPy, Torch and CUDA RNG states Provides the information needed for a more faithful resume.
12.3 last.ckpt versus best.ckpt
last.ckpt is updated every epoch and represents the latest recoverable state. best.ckpt is replaced only when validation ROC-AUC improves. This separation is important because the latest epoch is not necessarily the best-generalizing epoch.
12.4 Full training function
train_experiment creates mode-specific folders, builds datasets and loaders, constructs the appropriate ablation model, computes training-only class weights, freezes and later partially unfreezes the backbone,

trains epoch by epoch, writes history atomically, maintains last and best checkpoints, applies early stopping, reloads the best checkpoint, predicts the test set and saves metrics and predictions.

13. Metrics, Figures, Video-Level Evaluation and Ablation
Metric Interpretation
Accuracy Overall proportion of correct decisions.
Balanced accuracy Average recall across classes; more informative under imbalance.
Precision Among predicted fake samples, the proportion truly fake.
Recall Among true fake samples, the proportion detected.
F1 Harmonic mean of precision and recall.
ROC-AUC Threshold-independent ranking performance.
Average precision Summary of the precision-recall curve, useful for the positive class.
The notebook generates loss and F1 curves, a confusion matrix, ROC and precision-recall curves, and a probability distribution figure. Every saved image is checked to ensure its shortest edge is at least 600

pixels, making the outputs suitable for reports.
Video-level evaluation averages frame/sample fake probabilities within each video. Aggregation can reduce noisy frame-level decisions because multiple observations contribute to one video score. The ablation modes allow the study to quantify the contribution of each feature family and the Swin branch under a shared training pipeline.
13.1 Why ablation classes/modes are important
Mode Scientific question
swin_only How well do learned deep visual features perform alone?
texture_only How well do handcrafted texture descriptors perform without images?
swin_lbp Does local binary texture improve Swin?
swin_lbp_glcm Does adding spatial co-occurrence improve the fusion?
swin_lbp_glcm_gabor Does orientation/frequency texture add value?
full Does adding multiscale wavelet information provide the strongest combined result?
14. Final Quality Gates
validate_run verifies that required checkpoints, epoch metrics, test metrics and predictions exist. It

checks
checkpoint keys, rejects NaN/Inf numerical metrics, confirms non-empty predictions and requires

sample_id uniqueness. These tests convert silent experiment corruption into explicit failures.
15. Technical Observations and Recommended Revisions
The following items are derived directly from the uploaded notebook and should be considered before presenting the implementation as final production code.

Observation	Recommended action
Configured success statuses are not used	success_status_values is defined, but metadata filtering does not actually restrict rows by the status column. Add a normalized allowed-status filter if only successful ROI extractions should enter training.
Video-level threshold inconsistency	The test model uses the validation-selected threshold, but the video-level evaluation later uses CONFIG["threshold"] (0.50). Use the stored selected_threshold for consistent frame- and video-level decisions.
model_name is not dynamic	CONFIG contains model_name, but the class always constructs swin_v2_t. Either remove the key or implement a model factory.
Initial predict_loader threshold is overwritten	predict_loader creates predictions at 0.50, then train_experiment recalculates them using selected_threshold. This is harmless but redundant and can confuse readers.
epoch_threshold_f1 is unused	The best validation F1 returned by find_best_threshold is assigned but never logged. Add it to the epoch history or replace the variable with _.
last checkpoint metadata can lag	last_state is created before a potential best-AUC update, so its best_metric and best_threshold may reflect the previous epoch. Update the best state first or rebuild last_state afterward.
No true resume-training block	RNG, optimizer and scheduler states are saved, but train_experiment does not currently detect and restore last.ckpt. The checkpoint is resume-ready in content, but a loading branch is still needed.
No data augmentation	This is defensible because cached texture features must stay aligned with images, but it may limit generalization. An augmentation-aware feature extraction design could be explored.
Unused imports	dataclass, copy and some other imports appear unused. Removing them improves clarity.
Hard-coded ROI_ROOT	The root path is embedded in the metadata cell rather than CONFIG. Moving it into CONFIG would improve portability.

16. Overall Design Rationale
The notebook is structured as a reproducible experiment pipeline rather than a minimal training script. Its
main design choice is multimodal feature fusion: Swin V2 learns task-specific representations from eye images, while four handcrafted feature families encode interpretable texture statistics. The use of transfer
learning, differential learning rates, initial backbone freezing, partial unfreezing, class-weighted binary loss,

threshold tuning, early stopping and best-checkpoint selection is coherent with a moderately sized deepfake dataset and the risk of overfitting.

The custom Dataset and model classes are necessary because the experiment must keep two synchronized feature branches and support fair ablation modes. Epoch-based training is necessary because optimization is iterative; however, the code does not blindly run all 30 epochs. Validation ROC-

AUC, early stopping and checkpointing determine when training should stop and which epoch should be reported.

17. Notebook Cell-by-Cell Map

Cell	Responsibility
0	Experiment title, purpose and engineering standards.
1	local source package mount and dependency installation.
2	Search helper for locating metadata.csv.
3	Central CONFIG dictionary.
4	Imports, deterministic seed, device, run ID and output folders.
5	Atomic file-writing helpers.
6	Metadata normalization, file checks, duplicate control and video leakage assertions.
7	Dataset distribution figure.
8	LBP, GLCM, Gabor and wavelet extraction.
9	Versioned, resumable texture feature cache.
10	Training-only StandardScaler fitting.
11	Transforms, ablation feature groups, EyeFusionDataset and DataLoaders.
12	SwinTextureFusion model and freeze/unfreeze methods.
13	Metrics, threshold search, optimizer and checkpoint-state creation.
14	Shared epoch loop and prediction routine.
15	Smoke test that blocks full training on failure.
16	Complete experiment training, model selection and test evaluation.
17	Main experiment or all ablations execution.
18	Loss, F1, confusion matrix, ROC, PR and probability figures.
19	Video-level probability aggregation and metrics.
20	Ablation comparison figure.
21	Final file, checkpoint, numeric and prediction integrity checks.

18. Conclusion

The uploaded notebook provides a technically strong experimental foundation. Its most important strengths

are source-video leakage prevention, train-only normalization, versioned feature caching, atomic writes, transfer-learning control, best-model selection, ablation support and explicit quality gates. The identified

revisions do not invalidate the experiment, but addressing them would make the implementation clearer, more internally consistent and genuinely resumable.